DP1 2021-2022

Documento de Diseño del Sistema

Tyrants of the Underdark



- **Miembros**:

del Hoyo Carcaboso, David

Domínguez Ruíz, Andrés

Fernández Castillo, Javier

García Aguilar, Javier

Otero Barbasán, Manuel

Zarandieta Ortiz, David

GRUPO L7-4

- **Tutor:**

Müller Cejas, Carlos

Repositorio: https://github.com/gii-is-DP1/dp1-2022-2023-l7-4

Vídeo: https://youtu.be/h2opAOIM_Yg

V4

Índice

1. Historial de versiones 4

2. Introducción 5

3. Diagramas UML 12
   3.1. Diagrama de Dominio/Diseño 12
   3.2. Diagrama de Capas (incluyendo Controladores, Servicios y Repositorios) 17

4. Patrones de diseño y arquitectónicos aplicados 20
   - Patrón MVC 20
   - Patrón Decorador 20
   - Patrón Front Controller 21
   - Patrón Template View 21
   - Patrón Intercepting Filter 22
   - Patrón Repositorio 22
   - Patrón Servicio 23
   - Patrón Layer-Supertype 23
   - Patrón Modelo de dominio 24
   - Patrón Inyección de dependencias 24
   - Patrón Eager/Lazy Loading 25
   - Patrón Paginación 26
   - Patrón (Meta)Data Mapper 27
   - Patrón Proxy 28
   - Patrón Identify Field 29
   - Patrón indirección 30
   - Patrón plantilla 31

5. Decisiones de diseño 32
   5.1. MAPA Y POSICIONES 32
   Posiciones del tablero 32
   Pieza como entidad 32
   Adyacencias 33
   Plantillas 33
   Mapas personalizables. 34
   Mapa visual e interactivo 34
   Librería de Js para el mapa 34
   API para obtener un mapa en JSON 35

Despliegue de tropas blancas (Enemigos del juego) 36
5.2. CARTAS Y ACCIONES 37
Carta 37
Entidad de Acción 37
Uso de chain of responsability 37
Método de siguiente acción 37
Vistas dinámicas de acciones 38
5.3. PARTIDA Y GENERALES 39
-Reponer cartas en el mismo lugar: 39
Control de errores en creación 39
HUD 39
Vistas de partida 39
- Creación de una Auditoría 40
- Marcadores de control de ciudades 41
- Descomposición de servicio en servicios según funcionalidad o clases auxiliares: 42
5.4. RESUMIDAS 44
Entidad de Player o relacionar user con game 44
Implementación de Formatters en el uso de los controladores: 44
Posible atributo Outcast y Aspect 44
Interfaz grafica y Mapa interactivo como modulo extra 44
Agrupación de filtros para el listado de cartas 44
Contador de puntos de victoria 44
Control de recursos insuficientes (botón en gris) 45
Tutorial al inicio de partida vs tooltips 45

1. Historial de versiones

Fecha	Versión	Descripción de los cambios	Sprint
27/10/2022	V1	• Creación del documento	2
27/10/2022	V1.1	• Añadido diagrama de dominio/diseño • Explicación de la aplicación del patrón caché	2
05/11/2022	V1.2	• Decisión de diseño 1	2
16/11/2022	V2.0	• Decisión de diseño 2 ,3, 4, 5, 6 y 7	2
14/12/2022	V3.0	• Decisión de diseño 8 • Patrones de diseño MVC, Decorador y Cadena de responsabilidad • Diagrama de capas • Diagrama de dominio/diseño	3
13/01/2022	v4.0	• Patrones de diseño añadidos a partir de Cadena de responsabilidad • Decisiones de diseño: de la 8 a la 30 • Introducción Cambios en: • Diagrama de dominio • Diagrama de capas	4

2. Introducción

Este texto describe la complejidad a considerar para el desarrollo del juego de mesa Tiranos del Underdark. Durante el desarrollo del mismo, se muestran ciertos aspectos a tener en cuenta para el proceso de implementación del sistema. A continuación se desarrolla el contenido del juego:

Tiranos del Underdark es un juego competitivo para 2-4 jugadores que te sumerge en las historias de Dungeons and Dragons. ¿Quién será capaz de controlar las profundidades del Underdark?

Para ello contaremos con 4 de los clanes más importantes de la región. Cada jugador representará uno de estos clanes, y tendrá como objetivo crear el mazo más óptimo que le permita ser más eficiente durante su turno y hacerse con el control del mapa. Tendremos un mercado común, del cual todos podremos comprar cartas con puntos de influencia. Sé el primero en conseguir las mejores. Utiliza los puntos de poder para hacer acciones básicas. Intercambia 1 de poder para desplegar una tropa, 3 para devolver un espía enemigo y 3 para eliminar una tropa enemiga.





Cada carta contiene entre 1 y varias acciones de las siguientes:

- Acción de recurso (acciones que otorgan recursos durante tu turno. Por ejemplo: obtén +2 de influencia, obtén +1 de Poder)
- Acción inmediata (acciones que producen un efecto inmediato al jugarla. Por ejemplo: asesina 1 tropa, suplanta una tropa blanca)
- Acciones de final de turno (acciones que se resuelven justo al final del turno del jugador activo. Por ejemplo: al final del turno, asciende una carta jugada)
- Acciones complejas (una acción compleja se compone de varias de las acciones nombradas anteriormente o incluso de varias complejas. Tenemos 3 tipos: CHOOSE(Elige una entre varias opciones), THEN(si haces la primera acción entonces podrás hacer la segunda) y ALL(juega todas las acciones)).



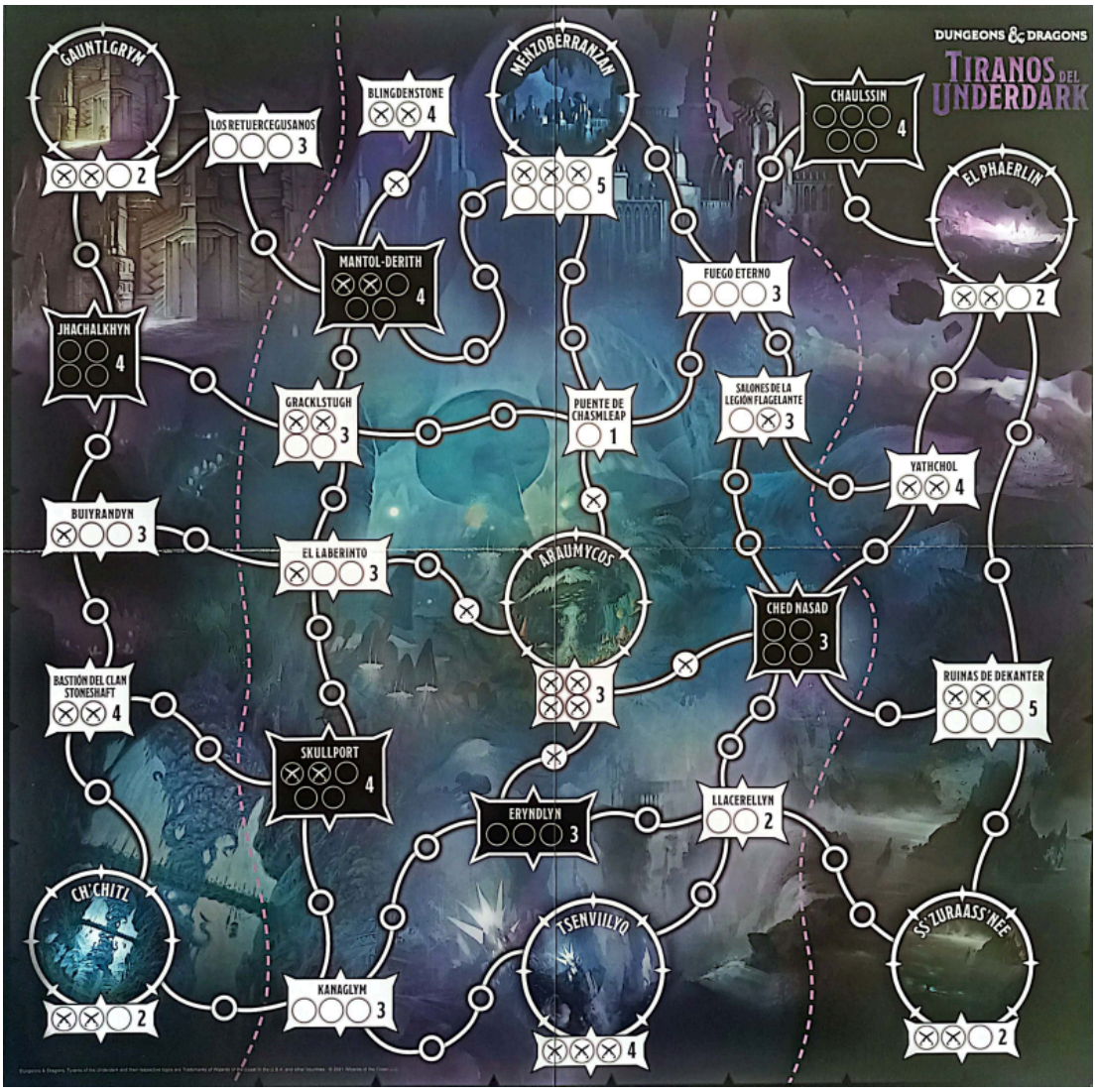






Controla las ciudades para obtener puntos de victoria. Conquista las capitales para obtener recursos y puntos de victoria adicionales durante el transcurso de las rondas.

Los espías cobran un valor clave en la estrategia de los jugadores. Lanza incursiones en zonas del mapa a las que tus tropas nunca llegarán. Tus tropas tendrán presencia en la ubicación en la que se encuentren y sus adyacentes, permitiendo desplegar tropas o asesinar tropas enemigas, entre otras cosas. Sin embargo, los espías podrán ser desplegados en la ubicación que quieras del mapa (exceptuando caminos -> solo en ciudades), y únicamente te otorgarán presencia en dicha ciudad, no en sus adyacentes.



Juega tus cartas de la forma más óptima, crea tu propia estrategia, compra las mejores cartas para llevar tu plan a cabo, controla el mayor número de ubicaciones posible y no dejes que nadie se interponga en tu camino. No te olvides de tu círculo interno, asciende cartas para obtener una mayor bonificación de puntos de victoria en el recuento de cartas de final de la partida.



Las condiciones de final de partida son una de las siguientes: todas las tropas de un jugador han sido desplegadas (40 tropas) o el mazo de mercado se ha agotado. Una vez se cumpla alguna de estas condiciones, se terminará la ronda de juego, asegurándonos de que todos los jugadores hayan jugado el mismo número de turnos, y se procederá al conteo de puntos de victoria. Puedes obtener puntos de victoria de la siguientes formas:

- Valor de PV de cada ubicación que controles
- 2 PV por cada ubicación bajo control total
- 1 PV por cada tropa en tu sala de trofeos (tropas enemigas asesinadas)
- Valor en PV de cada carta en tu mazo, mano y pila de descartes
- Valor en PV de cartas en tu círculo interno (cartas ascendidas)
- Fichas de PV obtenidas durante la partida





El jugador con más puntos de victoria será el ganador, y por tanto, el clan con más poder en el Underdark. ¿Serás capaz de afrontar el riesgo que esto conlleva o caerás en la oscuridad de las profundidades?

Los rasgos más destacables e interesantes de nuestro proyecto son la generación de mapas a través de adyacencias, resolución de acciones de cartas, acciones de final de turno, acciones condicionales y acciones de elegir, modificación (desplegar, asesinar, suplantar y devolver) del mapa a través de una interacción directa con el usuario, creación de mapas personalizado, interfaz dinámica e interactiva, selección del modo de despliegue de tropas blancas al iniciar una partida, creación de la partida restringiendo todo comportamiento negativo del sistema y creación de APIs para poder visualizar los mapas con una librería externa (D3) empleando javaScript y css.

Para su implementación se tienen en cuenta varias historias de usuario y módulos que permiten el correcto funcionamiento y rendimiento de dichas características.

- **Generación de mapas a través de adyacencias**: para generar un mapa de forma más automatizada se ha optado por crear un algoritmo que dado las ciudades y caminos que conectan dichas ciudades en un mapa, se generarán todas las posiciones necesarias para recorrer todos los caminos y todas la ciudades asociadas, generando así grafo.
- **Resolución de acciones de cartas:** para resolver las acciones que contiene una carta hemos decidido crear un algoritmo recorredor de acciones y subacciones. El algoritmo se encarga de mirar las acciones de una carta y resolverlas en orden (búsqueda en profundidad), recogiendo todo tipo de excepciones y saltos, y restringiendo cualquier posible punto muerto dentro de una partida.
- **Acciones de final de turno:** para poder abordar las acciones de final de turno se ha creado un tipo de acción llamada “END-TURN”, que permite añadir todas las acciones que estén bajo dicha acción a una lista de acciones que se ejecutarán y resolverán justo antes de pasar al siguiente jugador
- **Acciones condicionales:** las acciones condicionales (THEN) funcionan de la siguiente manera: Ejemplo (Asesina una tropa -> gana +1 de influencia). Este tipo permite hacer la segunda condición si la primera se ha realizado, de lo contrario, la segunda no se realizará
- **Acciones de elegir:** las acciones de elegir (CHOOSE) nos permiten seleccionar entre 2 o más opciones, según el número de veces que nos permita la acción. Una vez elegida la acción que queremos realizar, la otra parte del camino se bloqueará, resolviendo únicamente la elegida.

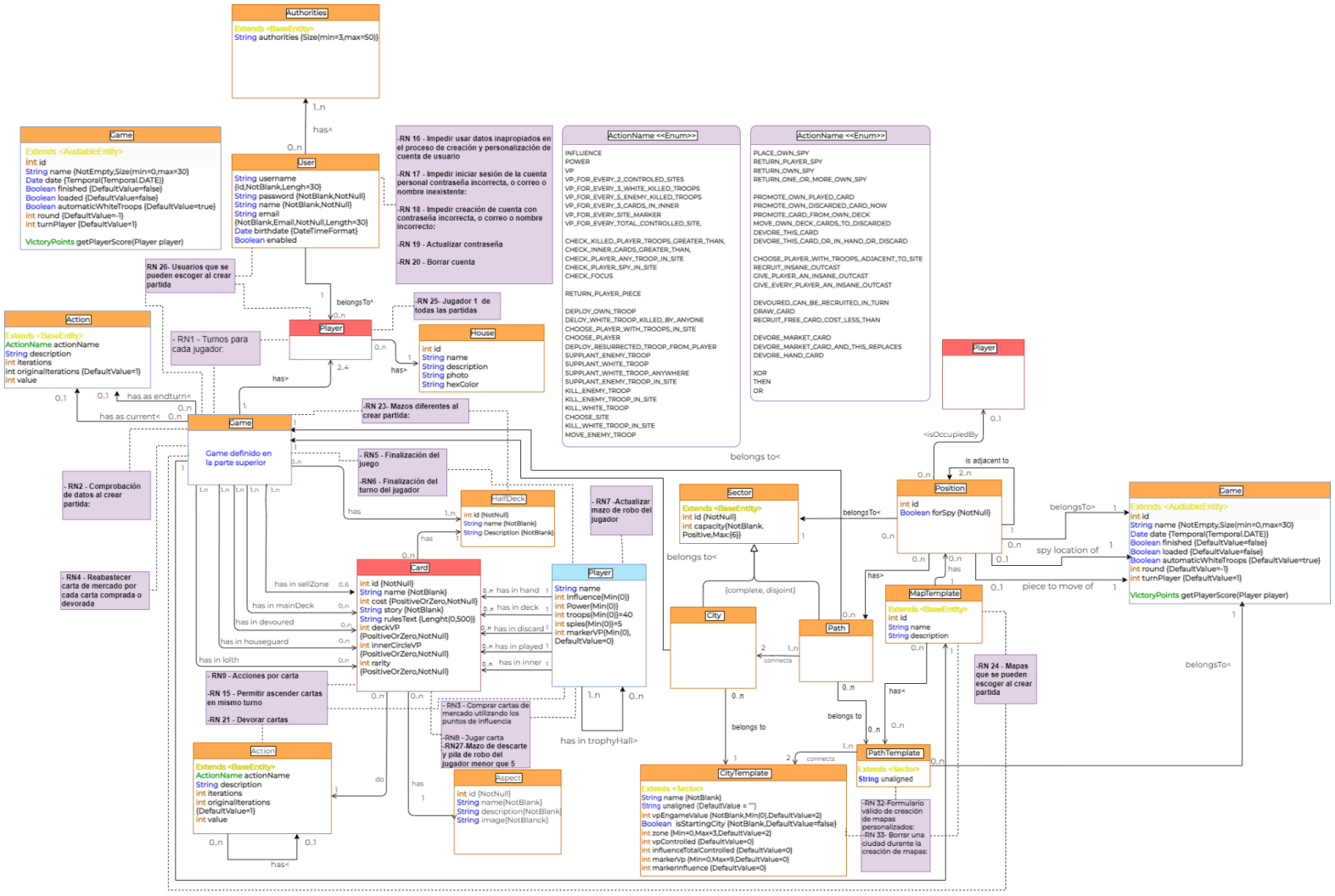
- **Modificación (desplegar, asesinar, suplantar y devolver) del mapa a través de una interacción directa con el usuario:** interactuar con el mapa da mucho valor a las decisiones del usuario, por ello damos la posibilidad de tener una interacción directa sobre lo que se hace. De esta forma evitamos errores y confusión a la hora de jugar. Cada tipo de acción calcula todas las posibilidades que tiene de ser ejecutada y se las ofrece al jugador como posibles opciones. Todas las posiciones disponibles siempre estarán visibles en el mapa.
- **Creación de mapas personalizado:** dar la posibilidad al usuario de tener sus propios mapas nos parecía una gran propuesta. Por ello tenemos un apartado dónde podemos crear mapas desde 0, visualizarlos o incluso editarlos. Para la correcta creación de un mapa, los formularios de creación notificarán al usuario si algo no se está introduciendo correctamente, impidiendo la creación de mapas erróneos.
- **Interfaz dinámica e interactiva:** la interfaz de juego muestra al usuario toda la información que necesita de manera visual. En todo momento, el jugador activo sabe lo que debe hacer, gracias a los mensajes en el layout superior, los recursos que tiene, y la pantalla le muestra la información relacionada con la parte del turno en el que se encuentre.
- **Selección del modo de despliegue de tropas blancas al iniciar una partida:** al crear una partida nueva y que el mapa seleccionado no tenga tropas blancas, tendremos la posibilidad, gracias a un algoritmo implementado, de desplegar el 28% de todas las posiciones generadas como tropas blancas de forma manual o automática. Esta opción la decidirá el usuario antes de empezar la partida.
- **Creación de la partida restringiendo todo comportamiento negativo del sistema:** al acceder a la creación de partida, tendremos múltiples opciones y parámetros que elegir antes de iniciarla. Para ello se ha optado por restringir todo tipo de comportamiento erróneo durante la creación. Únicamente podrás iniciar una partida si cumples con los requisitos de creación. El sistema no dejará seleccionar algunos parámetros dependiendo de otros (Ejemplo. tenemos 3 jugadores seleccionados, entonces al cambiar el mapa, no nos saldrá para seleccionar un mapa de 2 jugadores)

- **Creación de APIs para poder visualizar los mapas con una librería externa (D3) empleando javaScript y css**: se han creado 2 APIs para poder desplegar la visualización del mapa. Además se ha realizado un módulo extra con javaScript y CSS, que depende de una librería externa (D3) que nos permite trabajar con grafos, convirtiendo objetos a nodos y añadiendo aristas entre sí, en nuestro caso, dependiendo de las adyacencias.

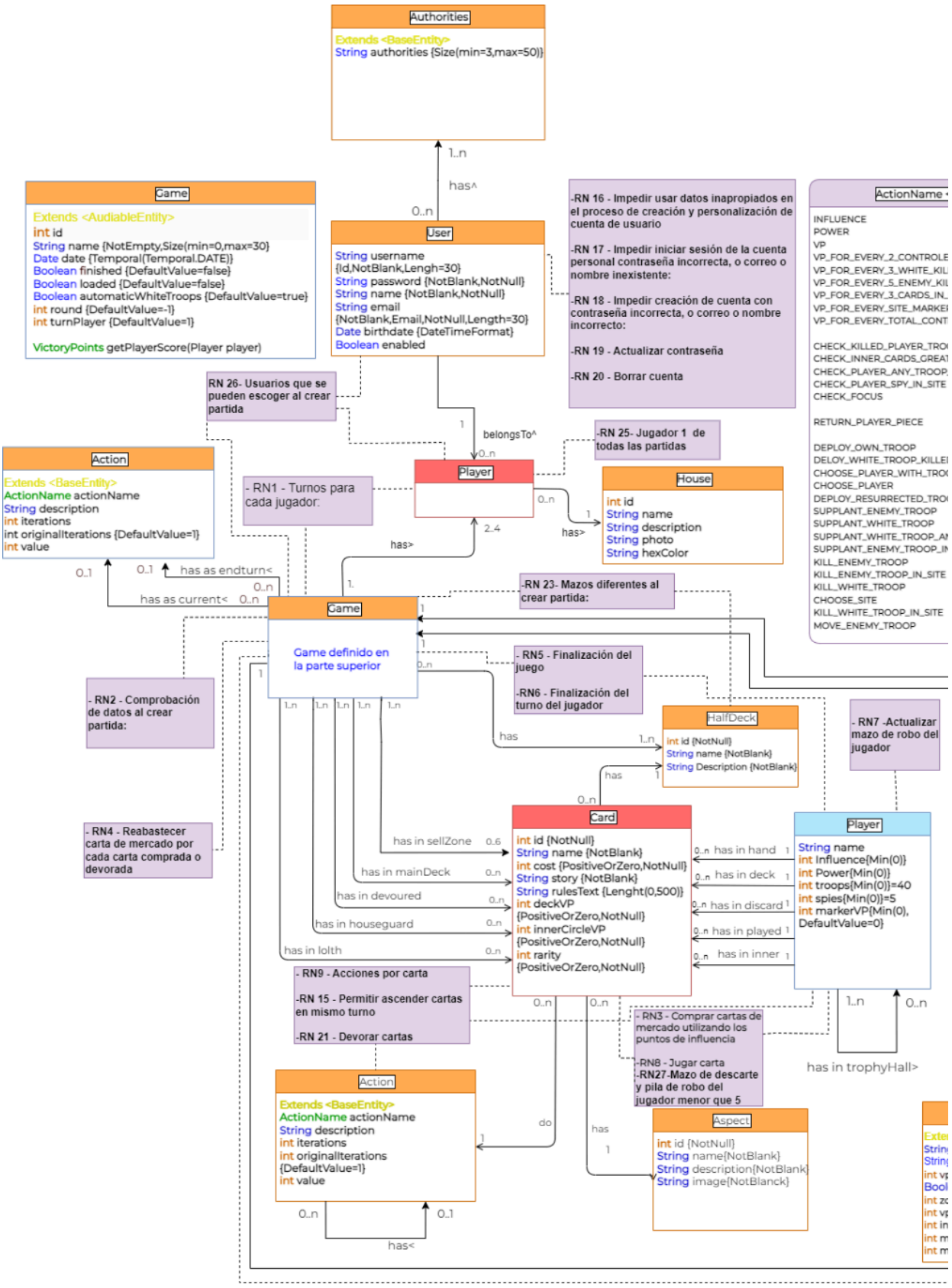
3. Diagramas UML

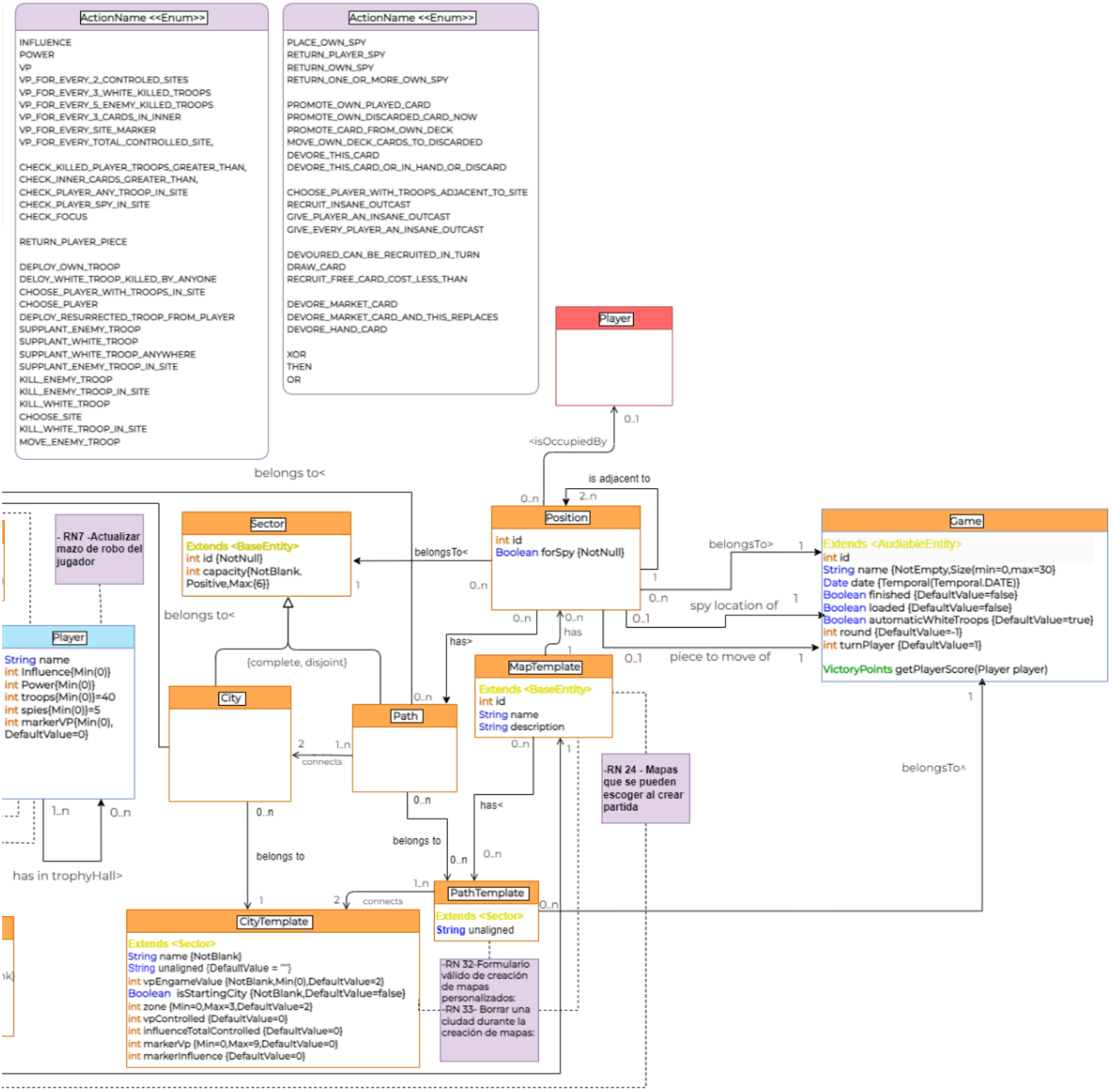
3.1. Diagrama de Dominio/Diseño

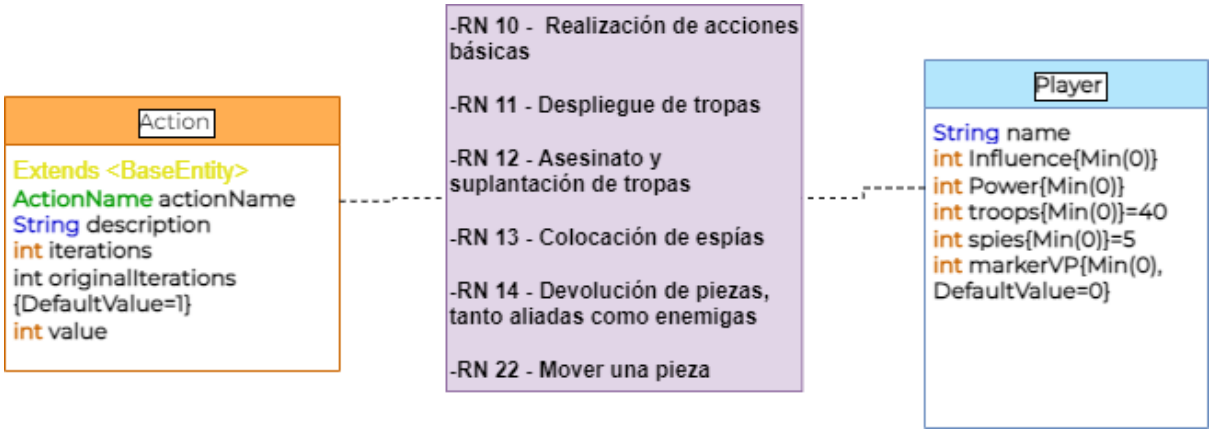
En esta sección podemos ver un diagrama UML de clases que describe el modelo de dominio para abordar la implementación del sistema requerido.

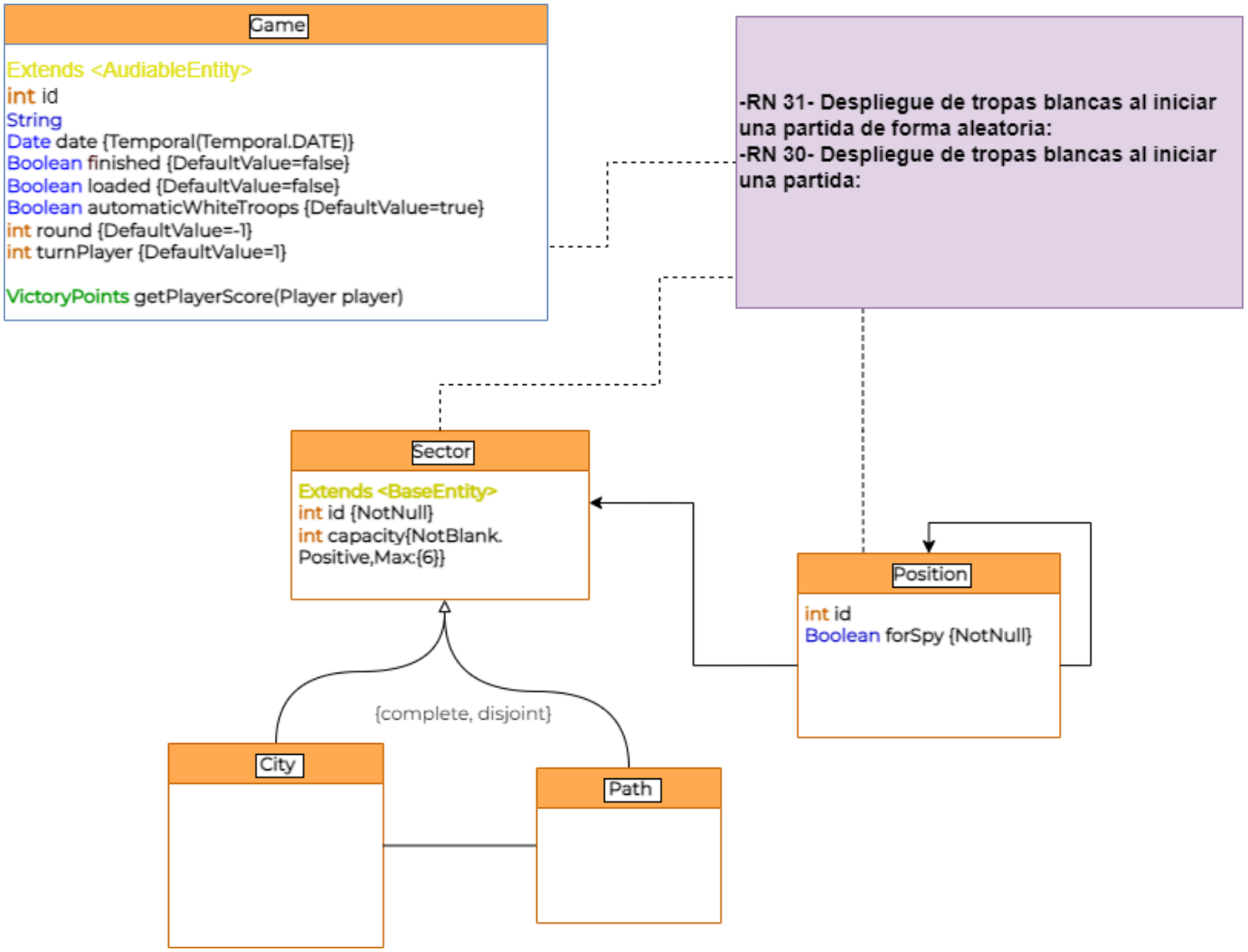


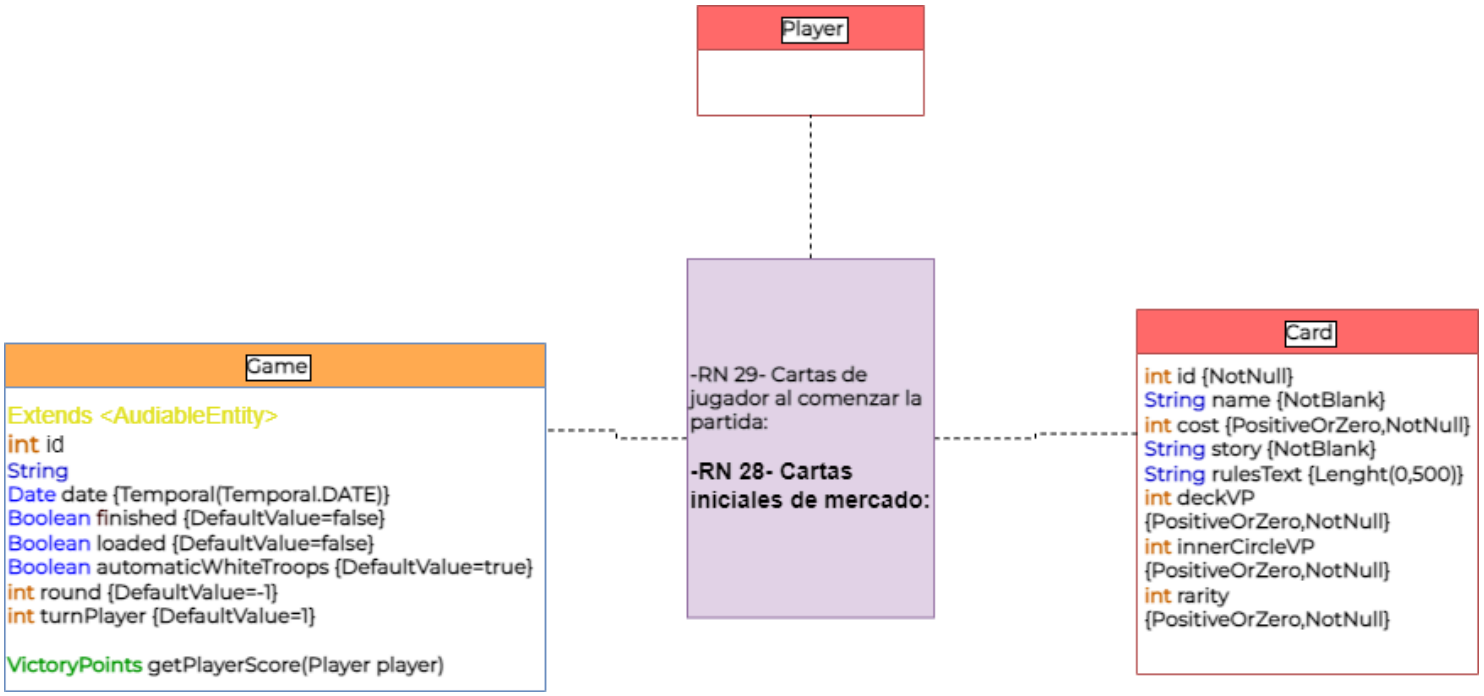
En las 2 próximas imágenes podemos ver el diagrama más grande, y en las otras próximas 3 se detallan algunas reglas de negocio que eran imposible de relacionar con sus entidades en el diagrama grande.


Authorities
Extends <BaseEntity>
String authorities {Size(min=3,max=50)}
1..n
has^
0..n
Game
Extends <AudiableEntity>
int id
String name {NotEmpty,Size(min=0,max=30)}
Date date {Temporal(Temporal.DATE)}
Boolean finished {DefaultValue=false}
Boolean loaded {DefaultValue=false}
Boolean automaticWhiteTroops {DefaultValue=true}
int round {DefaultValue=-1}
int turnPlayer {DefaultValue=1}
VictoryPoints getPlayerScore(Player player)
User
String username {Id,NotBlank,Lengh=30}
String password {NotBlank,NotNull}
String name {NotBlank,NotNull}
String email {NotBlank,Email,NotNull,Length=30}
Date birthdate {DateTimeFormat}
Boolean enabled
-RN 16 - Impedir usar datos inapropiados en el proceso de creación y personalización de cuenta de usuario
-RN 17 - Impedir iniciar sesión de la cuenta personal contraseña incorrecta, o correo o nombre inexistente:
-RN 18 - Impedir creación de cuenta con contraseña incorrecta, o correo o nombre incorrecto:
-RN 19 - Actualizar contraseña
-RN 20 - Borrar cuenta
ActionName
INFLUENCE
POWER
VP
VP_FOR_EVERY_2_CONTROLE
VP_FOR_EVERY_3_WHITE_KIL
VP_FOR_EVERY_5_ENEMY_KIL
VP_FOR_EVERY_3_CARDS_IN_
VP_FOR_EVERY_SITE_MARKE
VP_FOR_EVERY_TOTAL_CONT
CHECK_KILLED_PLAYER_TRO
CHECK_INNER_CARDS_GREA
CHECK_PLAYER_ANY_TROOP
CHECK_PLAYER_SPY_IN_SITE
CHECK_FOCUS
RETURN_PLAYER_PIECE
DEPLOY_OWN_TROOP
DELOY_WHITE_TROOP_KILLE
CHOOSE_PLAYER_WITH_TRO
CHOOSE_PLAYER
DEPLOY_RESURRECTED_TRO
SUPPLANT_ENEMY_TROOP
SUPPLANT_WHITE_TROOP
SUPPLANT_WHITE_TROOP_A
SUPPLANT_ENEMY_TROOP_IN
KILL_ENEMY_TROOP
KILL_ENEMY_TROOP_IN_SITE
KILL_WHITE_TROOP
CHOOSE_SITE
KILL_WHITE_TROOP_IN_SITE
MOVE_ENEMY_TROOP
RN 26- Usuarios que se pueden escoger al crear partida
belongsTo^
1
0..n
-RN 25- Jugador 1 de todas las partidas
Action
Extends <BaseEntity>
ActionName actionName
String description
int iterations
int originalIterations {DefaultValue=1}
int value
- RN1 - Turnos para cada jugador:
Player
House
int id
String name
String description
String photo
String hexColor
has>
2..4
0..1
has as endturn<
has as current<
-RN 23- Mazos diferentes al crear partida:
Game
Game definido en la parte superior
- RN5 - Finalización del juego
-RN6 - Finalización del turno del jugador
- RN2 - Comprobación de datos al crear partida:
HalfDeck
int id {NotNull}
String name {NotBlank}
String Description {NotBlank}
- RN7 -Actualizar mazo de robo del jugador
- RN4 - Reabastecer carta de mercado por cada carta comprada o devorada
Card
int id {NotNull}
String name {NotBlank}
int cost {PositiveOrZero,NotNull}
String story {NotBlank}
String rulesText {Lenght(0,500)}
int deckVP {PositiveOrZero,NotNull}
int innerCircleVP {PositiveOrZero,NotNull}
int rarity {PositiveOrZero,NotNull}
has in sellZone
has in mainDeck
has in devoured
has in houseguard
has in lolth
has in hand
has in deck
has in discard
has in played
has in inner
Player
String name
int Influence{Min(0)}
int Power{Min(0)}
int troops{Min(0)}=40
int spies{Min(0)}=5
int markerVP{Min(0), DefaultValue=0}
has in trophyHall>
- RN9 - Acciones por carta
-RN 15 - Permitir ascender cartas en mismo turno
-RN 21 - Devorar cartas
- RN3 - Comprar cartas de mercado utilizando los puntos de influencia
-RN8 - Jugar carta
-RN27-Mazo de descarte y pila de robo del jugador menor que 5
Action
Extends <BaseEntity>
ActionName actionName
String description
int iterations
int originalIterations {DefaultValue=1}
int value
do
has
Aspect
int id {NotNull}
String name{NotBlank}
String description{NotBlank}
String image{NotBlanck}
has<


ActionName <<Enum>>
INFLUENCE
POWER
VP
VP_FOR_EVERY_2_CONTROLED_SITES
VP_FOR_EVERY_3_WHITE_KILLED_TROOPS
VP_FOR_EVERY_5_ENEMY_KILLED_TROOPS
VP_FOR_EVERY_3_CARDS_IN_INNER
VP_FOR_EVERY_SITE_MARKER
VP_FOR_EVERY_TOTAL_CONTROLLED_SITE,
CHECK_KILLED_PLAYER_TROOPS_GREATER_THAN,
CHECK_INNER_CARDS_GREATER_THAN,
CHECK_PLAYER_ANY_TROOP_IN_SITE
CHECK_PLAYER_SPY_IN_SITE
CHECK_FOCUS
RETURN_PLAYER_PIECE
DEPLOY_OWN_TROOP
DELOY_WHITE_TROOP_KILLED_BY_ANYONE
CHOOSE_PLAYER_WITH_TROOPS_IN_SITE
CHOOSE_PLAYER
DEPLOY_RESURRECTED_TROOP_FROM_PLAYER
SUPPLANT_ENEMY_TROOP
SUPPLANT_WHITE_TROOP
SUPPLANT_WHITE_TROOP_ANYWHERE
SUPPLANT_ENEMY_TROOP_IN_SITE
KILL_ENEMY_TROOP
KILL_ENEMY_TROOP_IN_SITE
KILL_WHITE_TROOP
CHOOSE_SITE
KILL_WHITE_TROOP_IN_SITE
MOVE_ENEMY_TROOP
ActionName <<Enum>>
PLACE_OWN_SPY
RETURN_PLAYER_SPY
RETURN_OWN_SPY
RETURN_ONE_OR_MORE_OWN_SPY
PROMOTE_OWN_PLAYED_CARD
PROMOTE_OWN_DISCARDED_CARD_NOW
PROMOTE_CARD_FROM_OWN_DECK
MOVE_OWN_DECK_CARDS_TO_DISCARDED
DEVORE_THIS_CARD
DEVORE_THIS_CARD_OR_IN_HAND_OR_DISCARD
CHOOSE_PLAYER_WITH_TROOPS_ADJACENT_TO_SITE
RECRUIT_INSANE_OUTCAST
GIVE_PLAYER_AN_INSANE_OUTCAST
GIVE_EVERY_PLAYER_AN_INSANE_OUTCAST
DEVOURED_CAN_BE_RECRUITED_IN_TURN
DRAW_CARD
RECRUIT_FREE_CARD_COST_LESS_THAN
DEVORE_MARKET_CARD
DEVORE_MARKET_CARD_AND_THIS_REPLACES
DEVORE_HAND_CARD
XOR
THEN
OR
Player
0..1
<isOccupiedBy
belongs to<
is adjacent to
0..n
2..n
Position
int id
Boolean forSpy {NotNull}
Sector
Extends <BaseEntity>
int id {NotNull}
int capacity{NotBlank.
Positive,Max:{6}}
belongsTo<
1
0..n
belongsTo>
Game
Extends <AudiableEntity>
int id
String name {NotEmpty,Size(min=0,max=30}
Date date {Temporal(Temporal.DATE)}
Boolean finished {DefaultValue=false}
Boolean loaded {DefaultValue=false}
Boolean automaticWhiteTroops {DefaultValue=true}
int round {DefaultValue=-1}
int turnPlayer {DefaultValue=1}
VictoryPoints getPlayerScore(Player player)
spy location of
0..1
piece to move of
- RN7 -Actualizar mazo de robo del jugador
belongs to<
{complete, disjoint}
has>
has
MapTemplate
Extends <BaseEntity>
int id
String name
String description
Player
String name
int Influence{Min(0)}
int Power{Min(0)}
int troops{Min(0)}=40
int spies{Min(0)}=5
int markerVP{Min(0),
DefaultValue=0}
City
Path
2
1..n
connects
belongsTo^
-RN 24 - Mapas que se pueden escoger al crear partida
1..n
0..n
has in trophyHall>
belongs to
has<
PathTemplate
Extends <Sector>
String unaligned
CityTemplate
Extends <Sector>
String name {NotBlank}
String unaligned {DefaultValue = ""}
int vpEngameValue {NotBlank,Min(0),DefaultValue=2}
Boolean isStartingCity {NotBlank,DefaultValue=false}
int zone {Min=0,Max=3,DefaultValue=2}
int vpControlled {DefaultValue=0}
int influenceTotalControlled {DefaultValue=0}
int markerVp {Min=0,Max=9,DefaultValue=0}
int markerInfluence {DefaultValue=0}
-RN 32-Formulario válido de creación de mapas personalizados:
-RN 33- Borrar una ciudad durante la creación de mapas:


Action
Extends <BaseEntity>
ActionName actionName
String description
int iterations
int originalIterations
{DefaultValue=1}
int value
-RN 10 - Realización de acciones básicas
-RN 11 - Despliegue de tropas
-RN 12 - Asesinato y suplantación de tropas
-RN 13 - Colocación de espías
-RN 14 - Devolución de piezas, tanto aliadas como enemigas
-RN 22 - Mover una pieza
Player
String name
int Influence{Min(0)}
int Power{Min(0)}
int troops{Min(0)}=40
int spies{Min(0)}=5
int markerVP{Min(0), DefaultValue=0}


Game
Extends <AudiableEntity>
int id
String
Date date {Temporal(Temporal.DATE)}
Boolean finished {DefaultValue=false}
Boolean loaded {DefaultValue=false}
Boolean automaticWhiteTroops {DefaultValue=true}
int round {DefaultValue=-1}
int turnPlayer {DefaultValue=1}
VictoryPoints getPlayerScore(Player player)
-RN 31- Despliegue de tropas blancas al iniciar una partida de forma aleatoria:
-RN 30- Despliegue de tropas blancas al iniciar una partida:
Sector
Extends <BaseEntity>
int id {NotNull}
int capacity{NotBlank. Positive,Max:{6}}
Position
int id
Boolean forSpy {NotNull}
{complete, disjoint}
City
Path

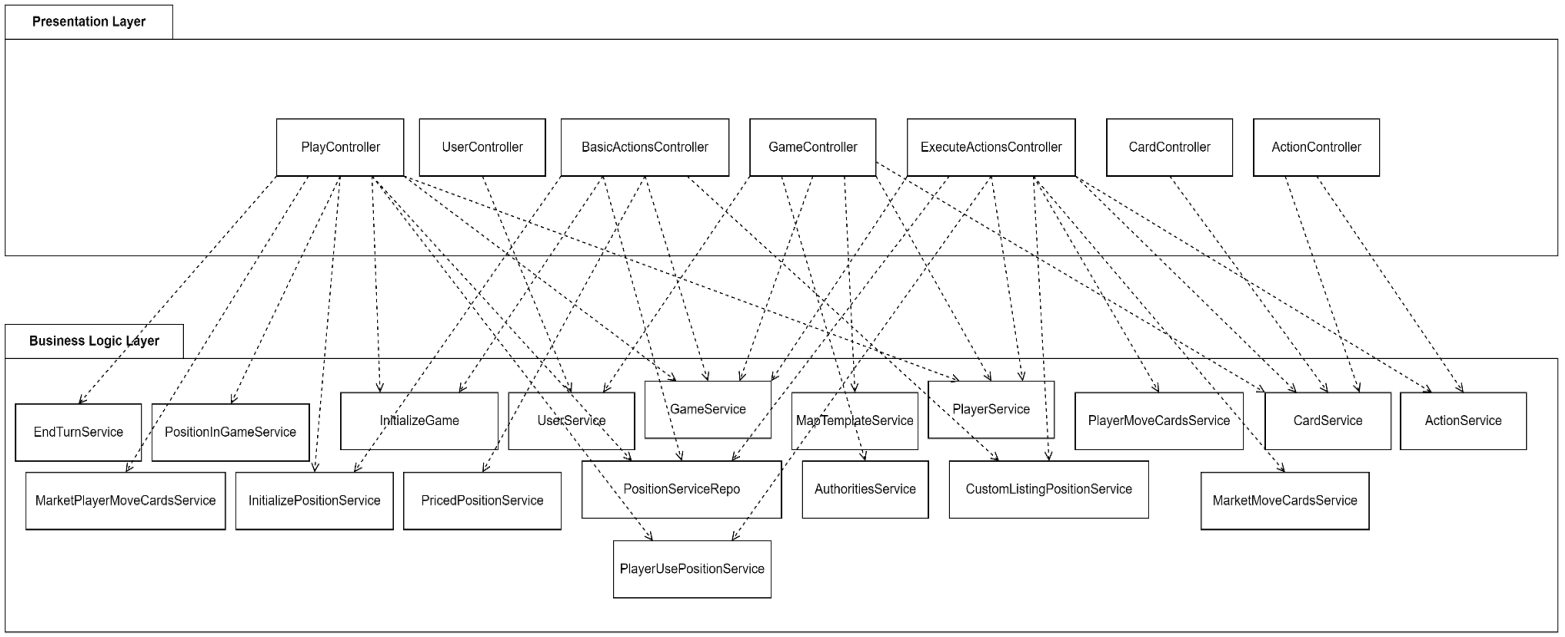

Player
Game
Extends <AudiableEntity>
int id
String
Date date {Temporal(Temporal.DATE)}
Boolean finished {DefaultValue=false}
Boolean loaded {DefaultValue=false}
Boolean automaticWhiteTroops {DefaultValue=true}
int round {DefaultValue=-1}
int turnPlayer {DefaultValue=1}
VictoryPoints getPlayerScore(Player player)
-RN 29- Cartas de jugador al comenzar la partida:
-RN 28- Cartas iniciales de mercado:
Card
int id {NotNull}
String name {NotBlank}
int cost {PositiveOrZero,NotNull}
String story {NotBlank}
String rulesText {Lenght(0,500)}
int deckVP {PositiveOrZero,NotNull}
int innerCircleVP {PositiveOrZero,NotNull}
int rarity {PositiveOrZero,NotNull}

3.2. Diagrama de Capas (incluyendo Controladores, Servicios y Repositorios)

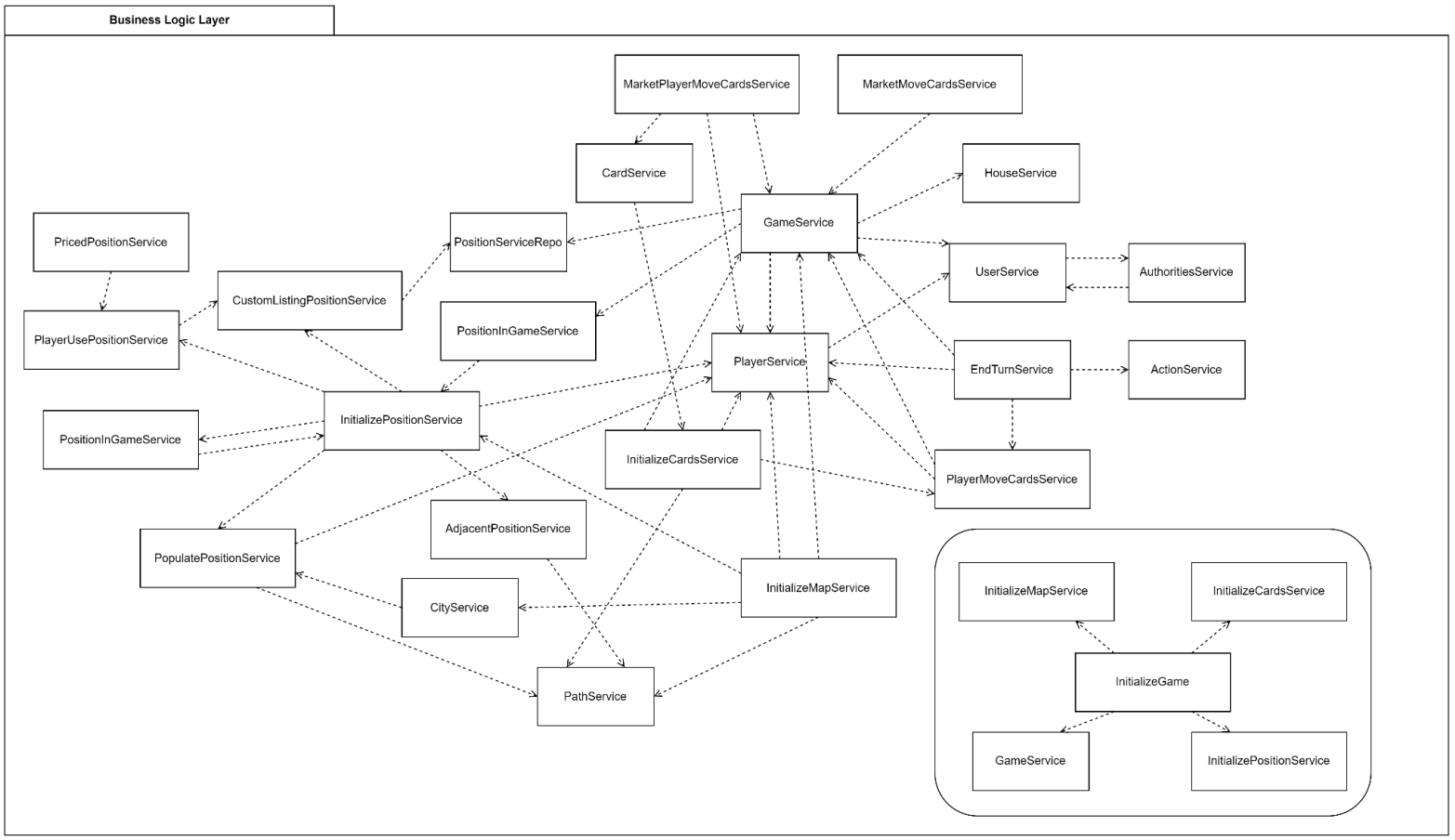
A continuación encontramos un diagrama UML de clases que describe el conjunto de controladores, servicios y repositorios implementados, incluída una división en capas del sistema como paquete horizontales:

Hemos realizado 3 subdiagramas que recogen las distintas clases pertenecientes a cada capa así como su relación entre ellas.

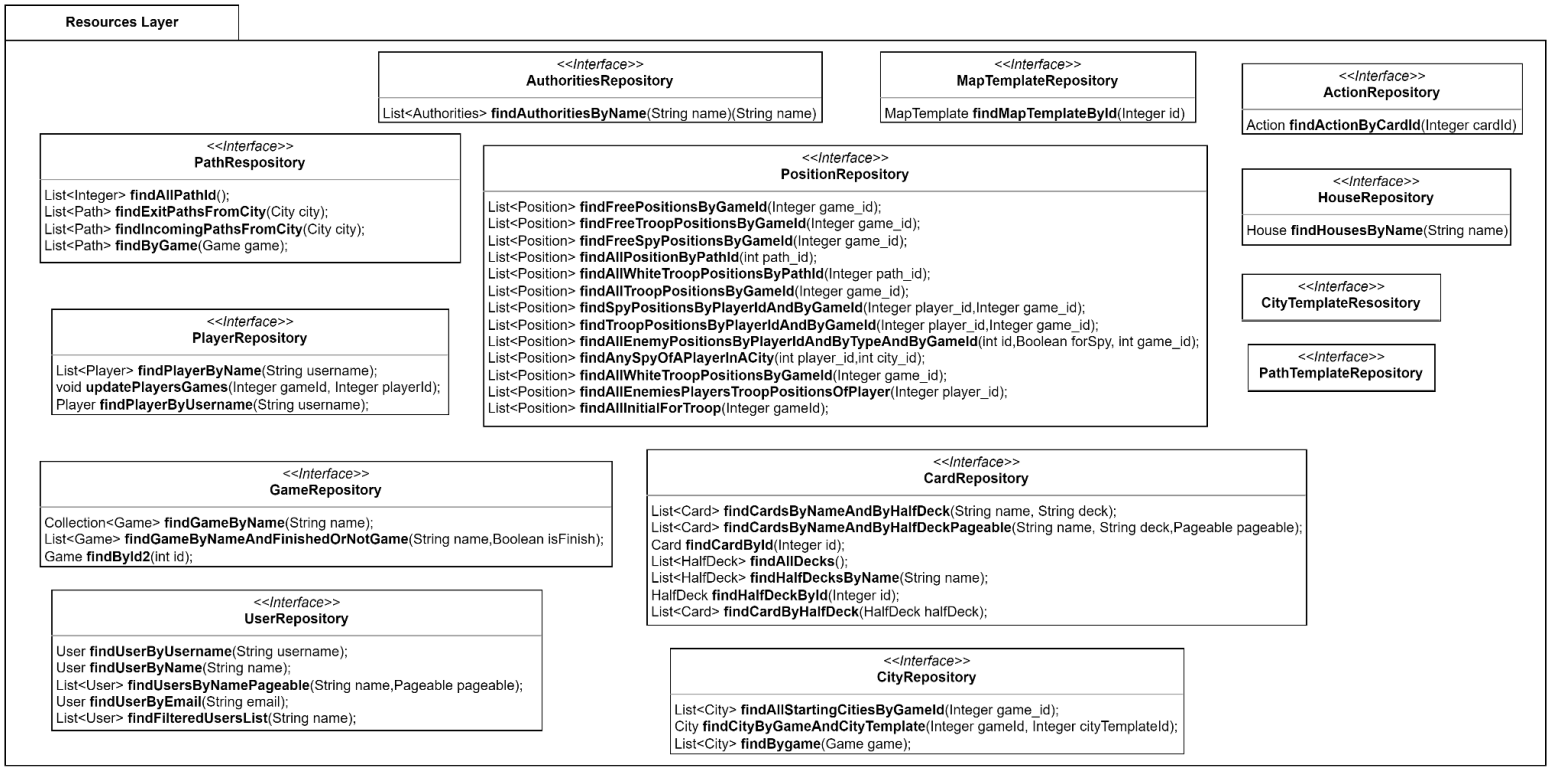
En la capa de presentación se incluyen todos los controladores que permiten la comunicación del usuario con el modelo del sistema, siendo algunos como PlayController, CardController,... Estos elementos interactúan con los servicios con los que están compuestos.



En la capa de Lógica de negocio se recogen todos los servicios del sistema, siendo algunos como PlayerUsePositionService,ActionService,... Estos servicios interactúan con los controladores que los necesiten y con los repositorios necesarios para aplicar la lógica del sistema.



En la capa de recursos se almacenan todos los repositorios del sistema, siendo algunos como PlayerRepository, PositionRepository,... .También se especifica las diferentes consultas personalizadas que hemos elaborado. Estos repositorios serán llamados por los servicios para obtener las entidades necesarias a los que se le aplicará la lógica de negocio implementado en los servicios. Cada clase de esta capa está relacionada con la capa de lógica de negocio en función del nombre de dicha clase, por ejemplo la clase PlayerRepository está relacionada con la clase PlayerService de la capa de lógica de negocio. Las clases CityTemplateRepository y PathTemplateRepository no tienen relación con la capa de lógica de negocio, ya que son utilizadas como plantilla para generar los elementos correspondientes en cada partida.



4. Patrones de diseño y arquitectónicos aplicados

En esta sección se especifica el conjunto de patrones de diseño y arquitectónicos aplicados durante el proyecto. Para especificar la aplicación de cada patrón puede usar la siguiente plantilla:

- **Patrón MVC**

Tipo: Arquitectónico

Contexto de Aplicación

Al utilizar SpringBoot tenemos clases de modelo, repositorios que representan a los datos que maneja el sistema, la vistas que son interfaces de usuario y los controladores que actúan como intermediario entre al vista y el controlador.

Clases o paquetes creados

Para todas las clases del sistema se ha creado un archivo Service y Controller asociado al modelo.

Ventajas alcanzadas al aplicar el patrón

Al aplicar este patrón obtenemos una clara separación de dónde tiene que ir cada tipo de lógica, facilitando el mantenimiento y escalabilidad, facilidad de testeo y reutilización de componentes.

- **Patrón Decorador**

Tipo: de Diseño

Contexto de Aplicación

Este patrón lo utilizamos para hacer más vistosas las cartas del juego mostrando todos sus atributos.

Clases o paquetes creados

Card.java, Halfdeck.java, Aspect.java y Action.java

Ventajas alcanzadas al aplicar el patrón

Combinamos varios comportamientos envolviendo un objeto con varios decoradores.

- **Patrón Front Controller**

Tipo: de Diseño

Contexto de Aplicación

Analiza la URL de las peticiones HTTP y lo redirige al módulo y método específico encargado de realizar la acción para satisfacer la petición.

Clases o paquetes creados

DispatcherServlet

Ventajas alcanzadas al aplicar el patrón

Permite controlar de forma centralizada y automática las peticiones que llegan. Permite añadir funcionalidades para que actúe de una forma u otra según las peticiones recibidas, por ejemplo si falta algún dato en un formulario puede volver a la interfaz para volver a pedir el dato faltante.

- **Patrón Template View**

Tipo: Arquitectónico

Contexto de Aplicación

Este patrón nos permite definir templates o plantillas para poder usarlas en aspectos de páginas como pueden ser el encabezado, el pie de página, el menú...

Clases o paquetes creados

En la carpeta WEB-INF aplicado en los archivos .jsp

Ventajas alcanzadas al aplicar el patrón

Nos deja aportar muy fácilmente una cohesión en lo visual de nuestra aplicación.

- **Patrón Intercepting Filter**

Tipo: Arquitectónico

Contexto de Aplicación

Permite añadir filtros de seguridad haciendo uso de librerías para poder filtrar la información que se le permite ver al usuario según su rol. Por ejemplo, en nuestro caso solo el usuario con el rol de *admin* se le permite mostrarle el botón de borrar jugador.

Clases o paquetes creados

SecurityConfiguration.java

Ventajas alcanzadas al aplicar el patrón

Nos permite filtrar la información para que si hay datos sensibles no queden expuestos a cualquier usuario.

- **Patrón Repositorio**

Tipo: de Diseño

Contexto de Aplicación

Este patrón lo utilizamos en todas las clases de repositorio que tenemos, se encarga de agrupar todas las operaciones en una única clase que las gestiona.

Clases o paquetes creados

Existe un archivo ...Repository.java en cada carpeta de cada entidad dentro de la carpeta src/main/java/org/springframework/samples/tyrants.

Ventajas alcanzadas al aplicar el patrón

Nos ayuda a reducir el número de querys y a hacerlas personalizadas.

- **Patrón Servicio**

Tipo: de Diseño

Contexto de Aplicación

Este patrón lo utilizamos en todas las clases de service que tenemos, es una capa intermedia entre el repositorio y el controlador.

Clases o paquetes creados

Existe un archivo ...Service.java en cada carpeta de cada entidad dentro de la carpeta src/main/java/org/springframework/samples/tyrants.

Ventajas alcanzadas al aplicar el patrón

Nos ayuda a agrupar todas las funcionalidades necesarias, aparte que nos permite hacer servicios transaccionales.

- **Patrón Layer-Supertype**

Tipo: de Diseño

Contexto de Aplicación

Este patrón lo utilizamos para características comunes entre objetos. Nosotros utilizamos BaseEntity.

Clases o paquetes creados

Existe un archivo ....java en cada carpeta de cada entidad dentro de la carpeta src/main/java/org/springframework/samples/tyrants extendido a esta entidad.

Ventajas alcanzadas al aplicar el patrón

Nos ayuda a ahorrar tiempo al programar cualquier entidad que requiera de un Id que es lo que aporta BaseEntity a la entidad extendida.

- **Patrón Modelo de dominio**

Tipo: de Diseño

Contexto de Aplicación

Diagrama que recoge todas las entidades y los procedimientos para tratarlos, así como las herencias.

Clases o paquetes creados

Un diagrama para toda la aplicación.

Ventajas alcanzadas al aplicar el patrón

Este patrón nos ha permitido ver de un vistazo la aplicación de forma general. Permite una lógica compleja, relacionar entidades y es soportado por muchos frameworks.

- **Patrón Inyección de dependencias**

Tipo: de Diseño

Contexto de Aplicación

Patrón de diseño utilizado para reducir la dependencia entre clases. En lugar de que una clase dependa directamente de otra, se le proporciona una dependencia a través de una inyección de dependencia.

Clases o paquetes creados

Tanto clases Service.java como Controller.java. Un ejemplo de esto sería en la clase GameController.java donde la etiqueta “@Autowired” inyecta los servicios requeridos para ser utilizados en el controlador.

Ventajas alcanzadas al aplicar el patrón

- Reduce el acoplamiento entre clases: Al no depender directamente de otras clases, una clase se vuelve más reusable y fácil de modificar.
- Facilita el testing: Al no depender de otras clases, es más fácil probar una clase de forma aislada.
- Mejora la flexibilidad: Al no depender directamente de otras clases, es más fácil cambiar la implementación de una clase sin tener que modificar el código existente.

- **Patrón Eager/Lazy Loading**

Tipo: de Diseño

Contexto de Aplicación

En el patrón Eager Loading, los objetos se cargan en memoria de forma inmediata cuando se instancian, mientras que en el patrón Lazy Loading, los objetos se cargan en memoria solo cuando se accede a ellos por primera vez

Clases o paquetes creados

Todas aquellas entidades con relaciones con otras entidades. Un ejemplo donde hemos aplicado este patrón sería en la entidad Player.java donde varias relaciones ManyToOne correspondiente a el patrón Eager Loading, pero en la relación con la entidad de Game.java se ha cambiado a Lazy mediante la anotación FetchType.LAZY.

Ventajas alcanzadas al aplicar el patrón

El patrón Eager Loading tiene la ventaja de que todos los objetos se cargan en memoria de forma inmediata, lo que puede ser útil en aplicaciones donde es importante tener todos los objetos disponibles de forma rápida.

Por otro lado, el patrón Lazy Loading tiene la ventaja de que solo se cargan en memoria los objetos que se necesitan, lo que puede ayudar a reducir el consumo de memoria y recursos en aplicaciones grandes o con muchos objetos innecesarios.

- **Patrón Paginación**

Tipo: de Diseño

Contexto de Aplicación

Este patrón consiste en la división de un conjunto de resultados en distintas páginas o secciones, para facilitar su visualización y navegación.

Clases o paquetes creados

Dentro de CardRespository.java hemos realizado una Custom Query donde pasamos por parámetro el tipo Pagable, configurado en el servicio asociado a esa llamada en CardService.java

Ventajas alcanzadas al aplicar el patrón

- Mejora la usabilidad: Al dividir los resultados en distintas páginas, se facilita la navegación y la visualización de los resultados para el usuario.
- Reduce la carga del servidor: Al dividir los resultados en páginas, se reduce la cantidad de datos que se deben procesar y enviar al cliente en una sola solicitud, lo que puede mejorar el rendimiento del servidor y evitar sobrecargarlo con grandes cantidades de datos.
- Ahorra ancho de banda: Al dividir los resultados en páginas, se reduce la cantidad de datos que se deben transferir a través de la red, lo que puede ahorrar ancho de banda y mejorar la velocidad de carga de la página.
- Facilita el control de datos: Al dividir los resultados en páginas, es más fácil controlar y gestionar la cantidad de datos que se muestran al usuario en cada momento, lo que puede ser útil para limitar el uso de recursos y evitar problemas de rendimiento.

- **Patrón (Meta)Data Mapper**

Tipo: de Diseño

Contexto de Aplicación

El patrón Data Mapper es un patrón de diseño que se utiliza para separar la lógica de persistencia de los datos de la lógica de negocio de una aplicación. Según este patrón, se crean dos clases, una clase de entidad que representa los datos y una clase de mapper que se encarga de mapear los datos entre la entidad y la base de datos, en nuestro caso la clase repository .

Clases o paquetes creados

Todos las entidades representadas en la base de datos junto con su clase Repository.java correspondiente. Un ejemplo sería la entidad User.java que contiene los atributos que representan en los campos de la tabla de usuario y UserRepository.java que contiene la lógica para acceder a la base de datos.

Ventajas alcanzadas al aplicar el patrón

- Separación de responsabilidades: El patrón Data Mapper permite separar la lógica de persistencia de los datos de la lógica de negocio de la aplicación, lo que facilita la mantenibilidad y extensibilidad del código.
- Independencia de la base de datos: El patrón Data Mapper permite cambiar la base de datos utilizada sin afectar a la lógica de negocio de la aplicación, ya que el mapper se encarga de realizar el mapeo entre la entidad y la base de datos.
- Mejora del rendimiento: El patrón Data Mapper permite optimizar el acceso a la base de datos, ya que el mapper puede realizar operaciones batch y utilizar mecanismos de cacheado para reducir el número de consultas a la base de datos.
- Facilita la integración con frameworks de persistencia: El patrón Data Mapper es compatible con frameworks de persistencia como Hibernate o JPA, lo que facilita la integración con estos marcos.

- **Patrón Proxy**

Tipo: de Diseño

Contexto de Aplicación

Patrón de diseño que se utiliza para proporcionar una interfaz de acceso a un objeto real, de forma que se puedan añadir funcionalidades adicionales sin modificar el objeto real.

Clases o paquetes creados

Todas las clases Service.java ya que en algunas de las funciones implementadas, se realizan más pasos además de llamar a la base de datos para obtener la información. Un ejemplo de este patrón sería en la función de saveUser dentro de UserService.java donde además de realizar la llamada a la base de datos, se establece el rol del usuario.

Ventajas alcanzadas al aplicar el patrón

- Control de acceso: El patrón Proxy permite controlar el acceso al objeto real, ya que el proxy puede añadir lógica para validar o autenticar al usuario antes de permitir el acceso al objeto real.
- Optimización: El patrón Proxy permite optimizar el uso del objeto real, ya que el proxy puede añadir lógica para cacheado o paginación para reducir el número de llamadas al objeto real.
- Extensibilidad: El patrón Proxy permite extender la funcionalidad del objeto real sin modificar su código, ya que se pueden añadir funcionalidades adicionales en el proxy.
- Ocultamiento: El patrón Proxy permite ocultar la implementación del objeto real detrás de la interfaz del proxy, lo que facilita el cambio de implementación del objeto real sin afectar a los clientes.

- **Patrón Identify Field**

Tipo: de Diseño

Contexto de Aplicación

El patrón Identify Field es un patrón de diseño que se utiliza para asignar un identificador único a cada instancia de una clase. Según este patrón, se crea una clase con un atributo id que es utilizado para almacenar el identificador único de cada instancia. El valor de id debe ser generado de forma automática y no debe ser modificado una vez asignado.

Clases o paquetes creados

Todas las clases entidades, con la etiqueta “@Entity”,que poseen un atributo id con la etiqueta “@Id”. Un ejemplo de este patrón aparece en la clase Card.java donde aparece el atributo id con la etiqueta correspondiente y a la etiqueta “@GeneratedValue” para generar un id nuevo a cada carta nueva generada en la base de datos.

Ventajas alcanzadas al aplicar el patrón

- Identificación única: El patrón Identify Field permite asignar un identificador único a cada instancia de una clase, lo que facilita la identificación y recuperación de estas instancias.
- Mejora del rendimiento: El patrón Identify Field permite optimizar el rendimiento al utilizar índices basados en el valor del identificador para recuperar las instancias más rápidamente.
- Facilita la integración con frameworks de persistencia: El patrón Identify Field es compatible con frameworks de persistencia como Hibernate o JPA, lo que facilita la integración con estos marcos.
- Facilita la integración con sistemas externos: El patrón Identify Field permite integrar fácilmente la aplicación con sistemas externos al utilizar el identificador único como clave de integración.

- **Patrón indirección**

Tipo: de Diseño

Contexto de Aplicación

Este patrón permite asignar la responsabilidad a un objeto intermedio que medie entre otros componentes para que no estén acoplados directamente, el intermediario crea un indirección.

Clases o paquetes creados

En ExecuteAction Controller.java, toma los valores enumerados de ActionName y los redirecciona a la vista correspondiente en función de la acción de dicha carta

Ventajas alcanzadas al aplicar el patrón

- Mejora la legibilidad del código al utilizar nombres significativos en lugar de valores numéricos o de cadena.
- Ayuda a prevenir errores al limitar las opciones posibles.
- Facilita la extensión del código, ya que se pueden agregar nuevas opciones o estados sin afectar el código existente.
- Ayuda a aislar la lógica de negocio de la lógica de presentación.

- **Patrón plantilla**

Tipo: de Diseño

Contexto de Aplicación

Este patrón te permite definir el esqueleto de una entidad o algoritmo en una clase base y permite a las subclases sobrescribir los pasos sin cambiar la estructura general del algoritmo.

Clases o paquetes creados

Utilizamos este patrón en las clases CityTemplate.java y PathTemplate.java para generar los caminos y ciudades en cada partida.

Ventajas alcanzadas al aplicar el patrón

- Reutilización de código: se puede evitar la duplicación de código al proporcionar una clase base que contiene la lógica común a varias entidades.
- Mejora la legibilidad del código: se pueden encapsular los detalles de implementación en la clase base, lo que permite una mayor claridad y legibilidad en las clases derivadas.
- Facilita la extensión del código: se pueden agregar nuevas entidades sin afectar el código existente al crear clases derivadas de la clase base.
- Ayuda a aislar la lógica de negocio: se pueden separar los aspectos generales de la lógica de negocio de los aspectos específicos de cada entidad.
- Facilita el proceso de depuración y mantenimiento del código: al centralizar la lógica común en una clase base, se puede reducir el número de lugares donde se deben hacer cambios en caso de un problema o mejora.

5. Decisiones de diseño

En esta sección se describen las decisiones de diseño que se han tomado a lo largo del desarrollo de la aplicación que vayan más allá de la mera aplicación de patrones de diseño o arquitectónicos. Se va a dividir en 4 secciones, las tres primeras hacen referencia al elemento del juego al que pertenecen estas decisiones: Mapa y posiciones, carta y acciones, o partida.

Por último se exponen de forma resumida las decisiones que se han tomado que son también importantes pero menos complejas.

5.1. MAPA Y POSICIONES

El juego tiene por tablero un mapa que a efectos prácticos es un grafo no dirigido, cada una de las posiciones ocupadas por una pieza necesita tener en cuenta que posiciones son adyacentes a ella, ya que el juego permite hacer ciertas acciones como matar o suplantar tropas enemigas siempre que tengas una tropa propia adyacente. Procedemos a explicar qué decisiones se tomaron para implementar esta parte del juego.

Posiciones del tablero

Lo primero que nos pasa a la cabeza es ver como implementar las posiciones en el mapa

La **primera opción** era **meterlas manualmente en base de datos**, lo cual tiene como ventaja que simplifica mucho el código, sin embargo, encontramos que nuestro mapa tiene 244 posiciones entre tropas y espías y a cada posición hay que asignarle la ciudad o el camino al que pertenece entre otras cosas.

Como esto no era viable por el tiempo que se iba a tardar en **realizar se decidió realizar un algoritmo que dada la capacidad de un sector** (ciudad o camino) generaba las posiciones y le asignaba los parámetros necesarios antes de guardarlas en base de datos, esto aunque complejo, logra que puedas jugar una partida con tan solo tener que introducir las ciudades y caminos del juego. Más adelante esto fué una de las razones por las que llegamos a los mapas personalizados.

Pieza como entidad

El siguiente paso es como **ocupar esas posiciones**, en el origen, se decidió implementar una entidad Pieza, que representa las piezas reales que se usan en la partida, las piezas tenían un jugador y un color para diferenciarlas, más adelante durante una sesión de refactor nos dimos cuenta que los jugadores tienen asociada una Facción con un color concreto, y que si las posiciones guardaban el jugador que las ocupaba teníamos los mismos datos que si la ocupaba una pieza.

Luego finalmente la **pieza dejó de existir en el código y los jugadores ahora tienen un contador de tropas y espías**, que se resta cada vez que ocupan una posición. Lo bueno de hacer las posiciones generadas por el algoritmo anterior es que se generan 4 posiciones de tipo espía y N posiciones de tipo tropa, con esto si un jugador ocupa una posición se entiende que esa

pieza es espía o tropa dependiendo del tipo de la posición. Gracias a esto logramos quitar una entidad más del código y pasar a tener unos simples contadores en un jugador.

Adyacencias

Otro de los puntos importantes mencionados anteriormente fue el de las **adyacencias**, los jugadores ya pueden desplegar tropas y poner espías pero falta asignar adyacencias para poder matar tropas o devolver espías. En este problema nos volvemos a encontrar con dos alternativas:

Poner las adyacencias de forma manual sería tan poco complejo como tedioso, ya que haciendo unos cálculos rápidos se acercaban a las 1000 adyacencias, por lo que esta alternativa se descartó.

Otra forma de hacerlos sería a través de un algoritmo que calcula las adyacencias. Lo cual es complejo pero te permite no tener que preocuparte por las adyacencias, este método debía implementar las restricciones necesarias del juego, cosa que es mucho más sencilla de hacer generalizando que teniéndolo en cuenta adyacencia a adyacencia.

La solución adoptada **fue este método de generación de adyacencias**, que está basado en la teoría de grafos de las Matemáticas Discretas. En nuestro caso no se usó una librería porque nuestras adyacencias se diferencian respecto a unas posiciones y además las clases que usamos en el modelo están especialmente creadas para trabajar con Spring, por esto se decidió implementar el método de forma manual empleando estos conocimientos adquiridos en otras asignaturas.

El método que calcula las adyacencias recorre todas las posiciones de una ciudad y las hace adyacentes entre ellas y a la primera posición de un camino que toque la ciudad. Los espías son los únicos que no son adyacentes a los caminos, ya que en el lore del juego se entiende que están infiltrados en la ciudad ocultos. Por último las posiciones de los caminos son adyacentes en línea con su anterior y su siguiente.

Plantillas

Una vez generado el mapa, nos dimos cuenta de que había un problema, las posiciones no estaban asociadas a un juego luego solo se podía jugar a un mapa en una única partida, para resolver esto planteamos las siguientes opciones:
Meter más mapas y que hubiera un único mapa por juego, lo cual resolvía el problema pero solo una partida tendría el mapa original, lo cual no nos parecía correcto.

Utilizar los mapas (ciudades y caminos) como plantilla y que por cada juego se generarán las respectivas posiciones. Esto suponía cambiar el algoritmo de generación de posiciones y adyacencias, lo cual era arriesgado por el nivel de complejidad de ambos.

Como solución decidimos usar las plantillas que nos parecieron más razonables, por cada juego se elegiría un mapa plantilla y con las ciudades y caminos que tenía este mapa se generaría el tablero de juego. Finalmente se logró implementar y esto dio lugar a plantearse uno de los módulos extras de este proyecto, los mapas personalizados.

Mapas personalizables.

Teniendo en cuenta la forma en la que habíamos hecho la generación de posiciones y adyacencias, con hacer un formulario que te permitiera meter ciudades y caminos podrás generar tus propios mapas. Esto era un auténtico *Game Changer* pero suponía añadir al proyecto restricciones nuevas y ver cómo podíamos hacer para que no fuera difícil de hacer para un jugador.

Cómo solución decidimos hacer un formulario que te permitiera tanto crear como editar mapas siempre que no estén jugando por alguien. Y decidimos que para ver que se creaba correctamente la mitad de la pantallas tuviera el grafo visual que es con el que se jugaría la partida anteriormente.

Mapa visual e interactivo

Para poder jugar, teniendo en cuenta las más de 100 posiciones y 1000 adyacencias, necesitábamos una forma de representar las posiciones del juego.

La primera opción fue usar una tabla, fácil de implementar y relativamente sencilla de jugar, porque tienes a tu alcance toda la información necesaria.
La otra opción era la de usar algún tipo de librería para representar grafos en html, lo cual sería visualmente mucho más atractivo, sería más intuitivo y fácil de ver lo que está pasando. Sin embargo esta decisión requiere de aprender las funciones de alguna librería de Javascript, que requiere muchas horas y no había certeza de lograr un resultado.

La solución que se adoptó fue bastante lógica, se decidió dejar la tabla para seguir desarrollando el juego y ver si más adelante en el proyecto se podía meter esta modificación, designando a un encargado de investigar este módulo. Durante el desarrollo del proyecto Manuel Otero fue investigando sobre las opciones y cómo implementarlo, y más adelante se acabó implementando.

Librería de Js para el mapa

El problema viene en que el mapa no se podía representar con html usando porque nuestro mapa no es una cuadrícula si no un grafo. Había que buscar una librería que permitiera representar grafos usando un *canvas* o un *svg* .
Existen múltiples opciones como chart.js o D3.js cada una con sus ventajas e inconvenientes. Para tomar una decisión se tomó en cuenta que la librería debería de ser de licencia abierta, relativamente sencilla y que resolviera nuestro problema.

Nos decantamos por D3.js que es una librería que lleva bastante en el mercado y aunque la documentación no es especialmente buena, existen ejemplos en la web. Concretamente necesitábamos lo que ellos denominan un *Network graph,* y

D3.js tenía una ventaja considerable y es que cuenta una simulación de fuerzas, que hace que los nodos se repelan unos a otros, lo que te permite no tener que meter coordenadas para los nodos. Aún así, para que los mapas con más ciudades se visualicen bien implementamos coordenadas en las ciudades y un método en los caminos que interpola las coordenadas de sus posiciones para que se forme un camino en línea recta entre las ciudades a las que llega, no teniendo así que meter manualmente las coordenadas de cada posición.

API para obtener un mapa en JSON

Para la visualización del mapa es necesario tener todas las posiciones de un juego, por lo tanto es necesario pasar a la vista tanto esos datos además de los datos necesarios como cartas o acciones que se realicen. Para resolver este problema teníamos dos opciones:

Pasar en todos los controladores el juego con sus posiciones además de la información necesaria y las posiciones seleccionables, lo cual dependiendo de la vista no era necesario y si hubiera que realizar algún cambio afectaría a más de medio proyecto.

Hacer una API que dado un id de un juego diera todo el contenido de un mapa, esto aunque requiera implementar una clase nueva, te permite ver un mapa en cualquier lugar, útil para poder ver el mapa en una vista de elegir acciones de carta o creación de un mapa personalizado.

Por la versatilidad que ofrecía la API y que además daba los datos en JSON algo muy sencillo de pasar a objetos Js que es lo que usa la librería D3, nos decantamos por usar la API.

Despliegue de tropas blancas (Enemigos del juego)

Necesitábamos establecer una forma de colocar las tropas blancas en el tablero. Debía de tenerse en cuenta que existen tropas blancas que tienen unas coordenadas concretas en el mapa original.

La primera solución era que todo mapa que se creara tuviera una serie de tropas blancas y cambiar el algoritmo de generación de posiciones para que las tuviera en cuenta. Esta opción quitaba el tener que poner tropas blancas antes de empezar una partida que puede ser tedioso en mapas grandes

La segunda solución era que el jugador inicial eligiera las tropas blancas, que aunque pueda ser pesado, le permite jugar exactamente como él quiera.

La última solución era que se calculará un porcentaje de posiciones ocupadas por tropas blancas que para el jugador es sencillo pero hay que tener en cuenta nuevas restricciones y cambiar el algoritmo de generación.

Se decidió implementar las tres opciones, si un mapa viene con tropas blancas se usan estas, en caso contrario se le permite elegir al jugador qué quiere hacer: Elegir las posiciones o que se elijan automáticamente. De esta forma nos aseguramos de que tanto los jugadores más casual como a los interesados por el juego, tengan una buena experiencia.

5.2. CARTAS Y ACCIONES

Carta

Se decidió que las cartas iban a estar referenciadas y que lo único en lo que se haría copia por cada juego son sus acciones.

Entidad de Acción

Las cartas aunque tengan muchos atributos, la mayoría son valores que aportan un precio o unos puntos para ganar la partida, lo más complejo son las acciones.

El problema de las acciones es que pueden contener subacciones, es decir, una carta puede decirte gana 5 de poder **Y** despliega una tropa. Los tipos de unión son Haz todo, elige una opción y Si haces x entonces puedes hacer y. Claramente esto nos da a entender que las acciones tienen forma de árbol, otra estructura de datos como los grafos.

Teniendo tanto acciones simples (sin hijos) como complejas, podríamos realizar dos clases distintas o una clase Action.java que tuviera los valores necesarios para una simple y además una Lista de subacciones.

Nos decantamos por usar un único tipo llamado Action.java, ya que nos parecía más sencillo trabajar con con un tipo que con dos. Además, diferenciar si una acción es simple o no, es tan fácil como preguntar si la lista de subacciones está vacía.

Uso de chain of responsability

A la hora de realizar las acciones, había que decidir una forma de recorrer ese árbol y elegir cuál era la siguiente acción. Se propuso el patrón de chain of responsability y se mandó a los encargados de realizar Action estudiarlo para ver si se podía aplicar.

Una vez estudiado llegamos a la conclusión de que este patrón necesita una serie de clases que se conectan unas a otras en forma de cadenas, y parte de nuestras acciones son redirecciones a una página para que el usuario elija lo que quiera hacer. Por tanto decidimos no usar este patrón y hacer un algoritmo que recorra el árbol en función del tipo de acción.

Método de siguiente acción

Para realizar el algoritmo, viendo que las clases no eran viables, se decidió hacer un método que recorre el árbol en profundidad.

Las acciones de tipo realiza TODAS dejan pasar a sus hijas y no se marcan como completadas hasta que todas sus hijas están completadas.

Las acciones de tipo ELIJE se devuelven a sí mismas para que el jugador elija, que lo que hace es marcar como completadas todas las subacciones de eleje que no sean la elegida, de esta forma, la siguiente vez que se pase por elige solo podrá bajar hacia la subacción elegida

Por último las acciones de tipo SI, ENTONCES funcionan como las del tipo TODAS, pero en el caso de que no se realice la primera hija con éxito, bloquea el resto y se marca a si misma como completadas

Un tipo de acción que salió más adelante es A FINAL DE TURNO que lo que hace es dar un juego copia todas sus hijas en un atributo de acciones de final de turno (que se lanzan a final del turno, ¿mágico verdad?). Una vez hecho esto se marca como completada a sí misma y llama recursivamente al método para que busque la acción que toque.

.

Vistas dinámicas de acciones

Para las acciones que requieren interacción por parte del jugador, era necesario hacer una vista. Para las que suman puntos se realizan de forma transparente para el jugador.

Para resolver este dilema, se propuso hacer una vista para cada acción, lo cual aunque muy personalizado requería mucho trabajo y en caso de un cambio en las acciones requeriría cambiar muchas vistas.
Otra opción que se propuso es hacer una o varias vistas generales que engloban todo, más complejo de diseñar pero tendría una UI más unificada.

Como solución se optó por implementar tres vistas distintas, una por cada tipo general de acciones. A la vista de elegir acción, con la carta jugada en el lateral, las opciones a elegir arriba y el mapa debajo, esto permite que tomes mejores decisiones.
Una vista de mapa, solo el mapa, para las acciones que cuestan poder y que no vienen de ninguna carta
Una vista donde elegir qué carta ascender, donde sale la carta jugada en el lateral y a continuación, todas las cartas que puedes ascender para que se elija una.

Atributos simples de cartas: Half Deck
Uno de esos atributos de carta que eran sencillos es el Mazo al que pertenece, podíamos realizar este mazo como una clase de java propia o como un enumerado. La clase requiere un poco más de trabajo ya que es una relación en la base de datos, el enumerado no guarda información sobre el mazo.
Como en el juego original existe un lote para cada mazo, es decir una historia y explicación de ventajas y desventajas del mazo, decidimos implementar el Half Deck como clase. Además, en un futuro si cada mazo necesitará más atributos como un color o similar, será fácil de añadir.

5.3. PARTIDA Y GENERALES

-Reponer cartas en el mismo lugar:

Debido a la funcionalidad de las listas de Java, al eliminar un elemento de una lista, los demás elementos modificaban su posición hacia la izquierda, esto supone que cuando se compran cartas del mercado, se mueven las cartas. Esto en el juego de mesa no ocurre, si compras una carta, el hueco queda donde quitaste la carta.

Para solucionar este "problema" entre muchas comillas, había que reponer una carta en el mismo índice(lugar de la lista) de donde se compró. Además si no quedan cartas para reponer, se debe de meter una carta invisible para que el tamaño de la lista sea siempre 6.

Esto aunque complica la forma de implementar el movimiento de cartas en el código, hace que la vista del juego se vea mucho más realista, por eso se decidió implementarlo.

Control de errores en creación

Se decidió que un jugador no pueda elegir nada en una partida que le impida el correcto funcionamiento del juego, cosas como jugar de 4 un mapa para 2 personas, cosa que en la vida real puedes hacer pero que decidimos bloquear en nuestro sistema

HUD

Para que el jugador tuviera toda la información posible y esta no ocupará mucho espacio, se puso la opción de poner botones a páginas nuevas con esa información más grande, pero esto hacía que el jugador no tuviese todo a mano, algo incómodo.
Como alternativa a esta solución, se propuso hacer unos logos en el lateral que indica la cantidad de recursos y puntos que posee el jugador. Aunque esta opción requiere un diseño de iconografía, vimos que era lo más recomendable para el correcto funcionamiento del juego.

Vistas de partida

Para las rondas de una partida sería el mismo problema que antes, usar varias páginas o condensar todo en una única vista. El problema de una única vista es que los elementos son pequeños y no se aprecian los detalles.

Como la segunda opción era la más lógica y ya se habían realizado iconos anteriormente, se decidió hacer unas imágenes simplificadas de las cartas, que solo contienen los elementos más importantes de ellas como el nombre, el precio o los puntos que da.
Para solucionar la información perdida, se decidió poner unos mensajes emergentes con la acción de la carta. Además antes de comprar una carta salta un popup en el que puedes ver la carta en mayor tamaño con toda su información.

- **Creación de una Auditoría**

Descripción del problema:

Para tener un mejor control de los datos que aparecen, se ha pensado en crear una auditoría sobre una de nuestras entidades.

Alternativas de solución evaluadas:

Alternativa A:

Aplicar auditoría a la entidad User, añadiendo como nuevos atributos la fecha de creación y la última fecha de modificación

-Ventajas:

- Posibilidad de crear métodos que te digan cuando un usuario está en línea o no según el tiempo que lleva utilizando su perfil
- Control constante de los datos de cada usuario
- Permite al administrador poder ver las fechas de creación y de modificación de los usuarios

-Desventajas:

- Más complicado de aplicar debido al identificador que utiliza la entidad

Alternativa B:

Aplicar auditoría a la entidad Game, añadiendo como nuevos atributos la fecha de última modificación de la partida, el creador de la partida y el último usuario que modificó la partida.

-Ventajas:

- Más fácil de aplicar
- Permite al administrador saber cuales son las partidas que se están ejecutando en este momento, quienes son sus creadores y quienes son los últimos usuarios que están modificando o jugando la partida

Justificación de la solución adoptada

Hemos decidido aplicar la auditoría sobre Game porque presenta mejores ventajas que la otra opción

Scoreboard

Secuencia de turno

Entidad martet

- **Marcadores de control de ciudades**

Descripción del problema:

Para saber qué jugador tiene control sobre una ciudad recompensas por control o control total existen una serie de marcadores que indican tanto la ciudad como la recompensa que obtiene el jugador que tiene el marcador. Este problema supone varias alternativas de solución

Alternativas de solución evaluadas:

Alternativa A:

Crear una entidad llamada Marker que esté asociada a las ciudades y a los jugadores.

-Ventajas:

- La entidad de City tendrá menos atributos
- Los jugadores tendrán un marcador de control por cada ciudad capital controlada
- Según el tipo de control que tenga el jugador sobre la ciudad, el marcador le otorgará distintas recompensar

-Desventajas:

- Más datos en el sistema
- Esta nueva entidad solo estaría asociada a las ciudades y a los jugadores

Alternativa B:

Añadir los atributos que hacen referencia a los tipos de control en las ciudades y crear métodos que devuelvan el jugador que controla y que controla totalmente en la ciudad.

-Ventajas:

- Se guardarán menos datos en la base de datos
- Permite a la UI representar menos datos
- La UI sólo tendrá que utilizar los métodos de las ciudades que calculan los jugadores que controlan para indicar quién tiene cada marcador.

-Desventajas:

- Métodos más complejos de controlar

Justificación de la solución adoptada
Hemos decidido escoger la opción B porque es la que permite al sistema gestionar una menor cantidad de datos de forma rápida y efectiva.

- **Descomposición de servicio en servicios según funcionalidad o clases auxiliares:**

Descripción del problema:
Para evitar acumular todos los métodos del servicio Position Service,se han planteado 2 opciones: dividir los métodos en servicios diferentes y eliminar completamente Position Service, o crear clases auxiliares para PositionService que tengan métodos según su funcionalidad.

Alternativas de solución evaluadas:

Alternativa A:
Descomponer PositionService en otros servicios

-Ventajas:

- No depender de un único servicio en el que se encuentran todos los métodos
- Uso de los servicios más legible para los demás
- División según funcionalidad:
  - ➢ PositionServiceRepo: métodos que llaman directamente al repositorio
  - ➢ CustomListingPositionService:métodos que realizan filtrados especiales sobre métodos del repositorio
  - ➢ PlayerUsePositionService:métodos que permiten al jugador interactuar con las posiciones
  - ➢ PricedPositionService:métodos que permiten al jugador interactuar con las posiciones según los recursos del jugador

-Desventajas:

- Alta complejidad al realizar los test de los servicios
- Se complica el mantenimiento de los servicios

Alternativa B:
Crear clases auxiliares para PositionService

-Ventajas:
- Más fácil de mantener y de realizar sus pruebas
- Más fácil de elaborar y de dividir los métodos de los servicios según funcionalidad

-Desventajas:
- Vuelves a depender de un único servicio
- El servicio tendrá métodos que realicen llamadas a estas clases auxiliares

Justificación de la solución adoptada:
La solución escogida es dividir PositionService en distintos servicios según sus funcionalidades, ya que evitamos tener todos los métodos en un único servicio.

5.4. RESUMIDAS

Guardar carta dentro de acción.
Esta decisión se tomó para poder mostrar la carta jugada cuando se realiza un acción. De esta forma el jugador sabe por donde va.

Entidad de Player o relacionar user con game

En princio se iba a relacionar cada usuario con un juego, pero para cada partida el usuario puede elegir una facción distinta y tendría puntos distintos. Por este motivo se decidión crear un jugador dentro de cada partida asociado al usuario.

Implementación de Formatters en el uso de los controladores:

Se decidió implementar formatters para la mayoría de entidades porque muchos de los controladores estaban sobrecargados con muchas funciones, esto nos permitía no llamar al servicio, y dejar que Spring lo maneje automáticamente, consiguiendo así que los controladores sean más entendibles. Esto es una de las decisiones que se tomó cuando se realizó el Refactor del código para seguir las buenas prácticas de programación.

Posible atributo Outcast y Aspect

Outcast son un tipo de carta que se usa en otros mazos que no están implementados, se ha dejado de tal forma que se pueda introducir en el futuro. Aspect es un atributo de carta, ciertas acciones de cartas de mazos no implementados requieren conocerlo, por tanto se ha dejado pero no se usa, por si en un futuro se implementan nuevos mazos.

Interfaz grafica y Mapa interactivo como modulo extra

Se decidió hacer estos módulos porque creemos que este juego es complejo de entender sin verlo, ya que no es una cuadricula sino un grafo y tiene un número elevado de posiciones.

Agrupación de filtros para el listado de cartas

Al abordar los filtros para el listado de las cartas, sugerimos la posibilidad de agruparlos juntos para hacer filtros de búsqueda avanzada en vez de hacer una vista para cada filtro, porque de esta forma minimizamos el número de vistas y facilita el uso para el usuario.

Contador de puntos de victoria

Se hace un desglose de los puntos de victoria en lugar de mostrar el total, requiere de una clase nueva que se ha creado pero le da mucha mas información al jugador.

Control de recursos insuficientes (botón en gris)

Cada vez que quieres comprar una carta, si la seleccionas la vez en mayor tamaño pero no puedes seleccionar el botón de comprar

Tutorial al inicio de partida vs tooltips

Decidimos que cuando pases el ratón por encima de cada elemento de la partida te salga una descripción del elemento. La otra opción era hacer un tutorial antes de empezar la partida, pero ya estan las normas del juego en el inicio y los tutoriales suelen ser molestos.